# $H^3$: Hybrid Architecture Using High Bandwidth Memory and High Bandwidth Flash for Cost-Efficient LLM Inference

Minho Ha, Euiseok Kim, and Hoshik Kim

***Abstract*—Large language model (LLM) inference requires massive memory capacity to process long sequences, posing a challenge due to the capacity limitations of high bandwidth memory (HBM). High bandwidth flash (HBF) is an emerging memory device based on NAND flash that offers HBM-comparable bandwidth with much larger capacity, but suffers from disadvantages such as longer access latency, lower write endurance, and higher power consumption. This paper proposes $H^3$, a hybrid architecture designed to effectively utilize both HBM and HBF by leveraging their respective strengths. By storing read-only data in HBF and other data in HBM, $H^3$-equipped systems can process more requests at once with the same number of GPUs than HBM-only systems, making $H^3$ suitable for gigantic read-only use cases in LLM inference, particularly those employing a shared pre-computed key-value cache. Simulation results show that a GPU system with $H^3$ achieves up to 2.69x higher throughput per power compared to a system with HBM-only. This result validates the cost-effectiveness of $H^3$ for handling LLM inference with gigantic read-only data.**

***Index Terms*—High bandwidth memory, high bandwidth flash, large language model.**

## I. Introduction

LARGE language models (LLMs) continue to evolve, moving from simple chatbots to research assistant, agentic AI, and multimodal AI [1]. Throughout this evolution, the sequence length required by LLM inference is increasing dramatically (e.g., the latest Llama 4 supports sequence lengths up to 10 M [2].). There is active research on efficiently processing such long sequences [3], [4] and on precomputed key-value (KV) caches for long sequence data shared by multiple requests [5]. In [5], when requests arrive, these pre-computed KV caches are read without writing. Processing extremely long sequences in this way during LLM inference requires large memory capacity and results in gigantic KV cache reads. For example, assuming Llama 3.1 405B [6] inference, the capacity of the shared pre-computed KV cache of 1 M and 10 M sizes is approximately 540 GB and 5.4 TB, respectively, requiring dozens of GPUs just to store these values.

From a hardware perspective, high bandwidth memory (HBM) capacity is insufficient compared to the increasing amount of data during LLM inference. To overcome this HBM capacity shortfall, high bandwidth flash (HBF), which vertically stacks NAND flash to mimic HBM, has been proposed [7]. HBF offers bandwidth similar to HBM and larger capacity than HBM, but suffers from disadvantages such as longer access latency, lower write endurance, and higher power consumption per bit than HBM. Therefore, sub-system architectures and use cases that can highlight the advantages of HBF while offsetting its disadvantages are needed.

Received 4 January 2026; revised 31 January 2026; accepted 1 February 2026. Date of publication 4 February 2026; date of current version 17 February 2026. *(Corresponding author: Minho Ha.)*

The authors are with SK hynix Inc., Icheon-Si 17336, South Korea (e-mail: minho1.ha@sk.com; euiseok.kim@sk.com; hoshik.kim@sk.com).

Digital Object Identifier 10.1109/LCA.2026.3660969

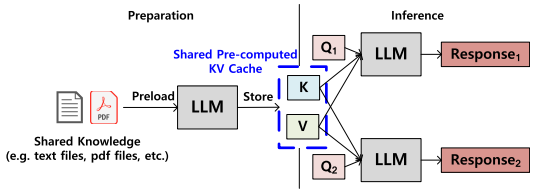



Fig. 1. The concept of cache augmented generation (CAG) [5], a representative example of a gigantic read-only use case.

In this paper, we propose a hybrid architecture for effectively utilizing HBF, considering its strengths and weaknesses. The key contributions of this paper are as follows:

- We propose $H^3$, a hybrid architecture that can effectively utilize HBM and HBF.
- We explore a use case well-suited for $H^3$.
- We demonstrate the cost-effectiveness of $H^3$ through a simulation-based performance evaluation.

## II. Motivation

### A. Gigantic Read-Only Use Case

During LLM inference, many data types exhibit read-only behavior. For example, LLM weight parameters are repeatedly read. Specifically, Llama 3.1 405B [6] requires approximately 405 GB to store model weights in FP8 precision. This means that 405 GB of reads occur each time a batch is processed.

Particularly, in cache-augmented generation (CAG) [5], which has recently attracted considerable research attention, when the LLM receives a query, it reads the gigantic shared pre-computed KV cache, performs a computation, and then outputs a response (Fig. 1). In other words, the shared pre-computed KV cache is inherently read-only. In CAG, attention computation uses shared KV attention, which amortizes memory access overhead, preventing significant latency increase even with a large batch size. Therefore, increasing the batch size can lead to substantial gains in throughput. However, due to the limited capacity of the HBM on GPUs [8], running CAG with large batch sizes is challenging. Storing the gigantic shared pre-computed KV cache on HBM requires more GPUs, increasing cost and overall system power requirements.

### B. High Bandwidth Flash

As shown in Fig. 2, HBF is a device that supports high bandwidth by vertically stacking NAND flash dies and

1556-6056 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: East China Normal University. Downloaded on October 19,2026 at 06:54:01 UTC from IEEE Xplore. Restrictions apply.

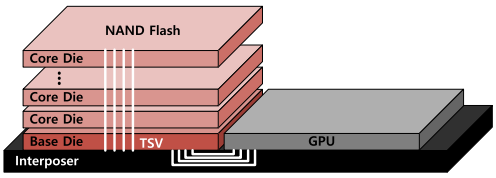



Fig. 2. The concept of HBF.

connecting the base die and NAND flash dies with through silicon vias (TSVs). Currently, various memory and flash vendors are developing HBF aiming to support bandwidth similar to HBM and much larger capacity than HBM [7]. Since HBF is not yet a commercially available device, in this paper, we will assume its target specifications [7]. The expected advantages of HBF are 1) up to 16x larger capacity than HBM, and 2) similar bandwidth to HBM, and the expected disadvantages are 1) slower access (ns vs. $\mu$s), 2) lower write endurance, and 3) up to 4x higher power consumption than HBM.

Since HBF has clear advantages and disadvantages, it can only be effectively used in certain use cases, and the gigantic read-only use case mentioned in Section II-A can be an effective use case. Gigantic KV cache data and weight parameters can be stored due to the large capacity of HBF, and the low write endurance of HBF is no longer a disadvantage because they have a read-only data pattern. We will discuss a way to overcome the slow access latency of HBF later in this paper.

## III. Proposed Architecture and Its Usage

### A. $H^3$: Hybrid Architecture Using HBM and HBF

As mentioned in Section II-B, since HBF is a device with both advantages and disadvantages, its use alone has limitations. Therefore, we propose $H^3$, a hybrid architecture that utilizes both HBM and HBF, aiming to highlight the advantages of HBF and compensate for its shortcomings. There can be various implementation methods for hybrid architectures. For example, one method places the HBM and HBF directly next to the GPU's shoreline. However, this method has the disadvantage of reducing the number of HBMs due to the limited shoreline space on the GPU. To maximize the bandwidth of the HBM and HBF, as shown in Fig. 3(a), the HBMs in the $H^3$ are connected to the GPU's shoreline, and the HBMs and HBFs are daisy-chained. Within the HBM base die, memory access is divided into two paths by address decoder & router: one accessing the HBM and the other accessing the HBF. Consequently, the GPU can directly access the HBF through the HBM base die (Fig. 3(b)). In other words, both the HBM and HBF serve as the GPU's main memory. Since both HBM and HBF are used as main memory, the host uses the unified address space with divided memory regions when accessing HBM or HBF (Fig. 3(c)).

In Fig. 3(b), it is assumed that the GPU, HBM base die, and HBF base die are connected via a die-to-die (D2D) interface and the bandwidth between the GPU and the HBM base die is equal to the bandwidth between each base die and the core die. We also assumed that the HBM and HBF controller are located on their own base die. This is subject to change depending on silicon area availability.

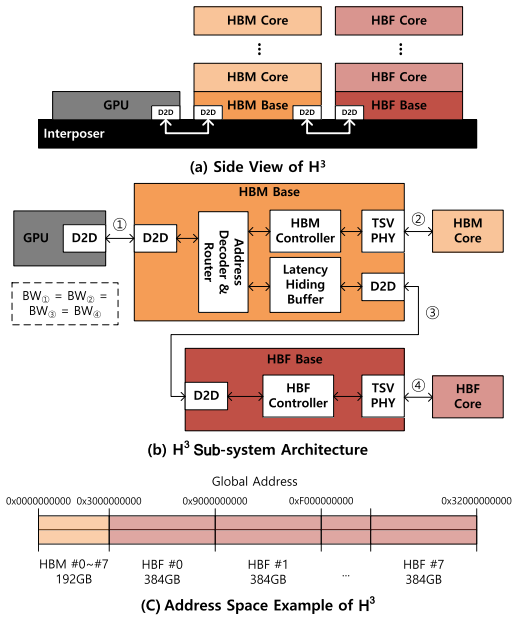



Fig. 3. $H^3$ Architecture overview.

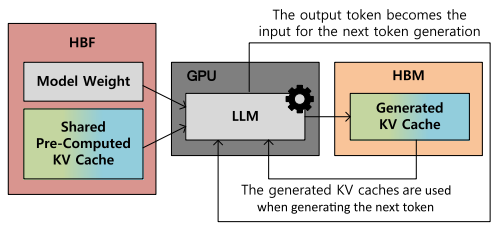



Fig. 4. A description of how HBM and HBF are accessed in LLM inference with gigantic shared pre-computed KV cache.

### B. Usage: LLM With Gigantic Read-Only Data

$H^3$ is suitable for LLM inference with the gigantic model weights and shared pre-computed KV cache described in Section II. These values, which are gigantic and read-only, are well-suited for the HBF supporting large capacity and having limited write endurance. Therefore, the model weights and shared pre-computed KV caches are stored in the HBF. In addition, the generated KV caches and other data are stored in the HBM. Fig. 4 explains how it works. In Fig. 4, when accessing HBF, hop latency may be added, but since LLM inference is memory bandwidth-bound, hop latency can be sufficiently hidden. Since the data access pattern of LLM inference is deterministic [9], we expect that it will be possible to allocate data suitable for the HBM and the HBF with the support of deep learning framework. Processing a particular operation may require data stored in both the HBM and HBF. However, because the tensors required for the operation can be accurately predicted, tensor-level scheduling is possible, making contention between the HBM and HBF manageable.

The advantage of applying $H^3$ to gigantic read-only use cases is that it supports larger capacity allowing for larger batch processing with the same number of GPUs. Larger batch processing leads to improved throughput, ultimately resulting in cost savings when running an LLM inference system. Using

Authorized licensed use limited to: East China Normal University. Downloaded on October 19,2026 at 06:54:01 UTC from IEEE Xplore. Restrictions apply.

HBF for read-only use cases also has the additional benefit of reducing garbage collection and wear-leveling overhead.

### C. Optimization: Latency Hiding Buffer

As HBF is inherently NAND flash-based, it has longer access latency than HBM. Therefore, hiding its long latency is essential for using it as memory. To compensate for the long NAND flash latency, the latency hiding buffer (LHB), which is a kind of prefetch buffer, is integrated into the base die of either HBM or HBF of the $H^3$ (In Fig. 3(b), LHB is in the base die of HBM). For LLM inference, the next layer can be processed only after the previous layer is completed, and the model weight and KV cache required for computation in one layer are pre-determined. In other words, LLM inference has a deterministic and sequential data pattern [9]. Therefore, the data required for computation can be accurately predicted in advance and the latency of NAND flash can be hidden by continuously bringing the data required for the computation to the LHB. To effectively utilize the LHB, the deep learning framework needs to be modified to provide prefetching hints. Since prefetching hints are deterministic and the prefetched data unit is at the coarse-grained tensor-level, the prefetching overhead is expected to be small. Expected size and silicon area overhead of the LHB will be discussed later.

## IV. Evaluation

### A. Experimental Setup

*1) Method:* As the HBF is a device under development, actual measurements are impossible. Therefore, an in-house simulator based on analytical modeling, proven in previous studies [3], [4], [10], is developed to evaluate the performance of $H^3$-equipped system. The in-house simulator estimates execution time, throughput, throughput per power consumption based on computation time, data transfer time, and communication time between GPUs. Computation time is calculated based on the number of operations of the LLM model and the GPU performance. Data transfer time of HBM and HBF is calculated based on the LLM model size, KV cache size, and bandwidth of devices. When calculating data transfer time, data is partitioned and duplicated to optimize performance by considering tensor parallelism (TP) and data parallelism. In addition, communication time between GPUs is calculated assuming ring all-reduce that occurs when applying TP.

*2) Workload:* Llama 3.1 405B [6] is selected as the workload for performance evaluation. The selected Llama 3.1 405B is sufficiently representative because it is large enough to be comparable to the latest frontier LLMs, and the pre-computed KV cache is related to the context window size but not to the model weights size. The original Llama 3.1 405B has a context window of 128 K, but to evaluate long sequence processing performance, we assume support for 1 M and 10 M. We assume that input sequence length (ISL) and output sequence length (OSL) are 1 K each, and the remainder except for ISL and OSL is occupied by the shared pre-computed KV cache. In the current experimental environment, the shared pre-computed KV cache occupies approximately 35% and 84% of the total capacity for 1 M and 10 M cases, respectively.

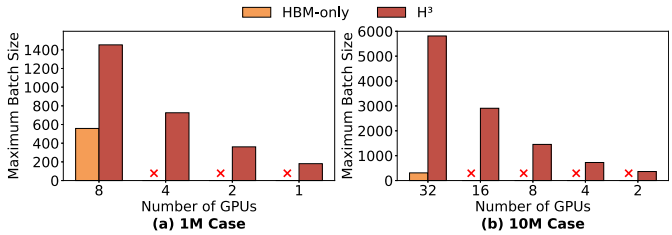



Fig. 5. Maximum batch size based on number of GPUs (x means that it cannot be run due to insufficient memory capacity.).

*3) HBF Specifications:* We assume the GPU is the B200, the latest GPU currently available [8]. In the B200, HBM3e, supporting 192 GB and 8 TB/s per GPU (24 GB and 1 TB/s per cube), is mounted. Based on the relative performance of HBF in Section II-B, we assume the HBF has a capacity of 3 TB, approximately 16x larger than the HBM3e, and a bandwidth of 8 TB/s, the same as the HBM3e. The thermal design power (TDP) per cube of HBM3e and HBF is assumed to be 40 W and 160 W, respectively, considering the per-cube capacity and the power consumption per bit of current NAND flash. Power estimation is based on current NAND flash technology. However, we expect that power consumption of HBF will be further reduced through power optimization during the HBF development process.

*4) Hardware Configuration:* We assume a DGX-style system [8]. Experiments with 1 M sequence length use 8 GPUs, while those with 10 M sequence length use 32 GPUs. These numbers of GPUs are the minimum number of GPUs required to run each sequence length for the HBM-only case. Therefore, in the 10 M case, communication between GPU servers occurs through a scale-out fabric (e.g., InfiniBand) rather than a scale-up fabric (e.g., NVLink).

### B. Experimental Results

*1) Batch Size:* By storing the gigantic shared pre-computed KV cache in the HBF, more free capacity is available in the HBM, allowing GPUs with $H^3$ to process larger batch sizes. Fig. 5 shows the processable batch sizes for the 1 M and 10 M cases, depending on the number of GPUs. In the 1 M case, shown in Fig. 5(a), when using $H^3$, batch sizes up to 2.6x larger can be processed than when using HBM alone. Similarly, in the 10 M case, shown in Fig. 5(b), using $H^3$ allows for batch sizes up to 18.8x larger than using HBM alone. There are various factors that contribute to improving throughput, but increasing batch size is clearly the most important factor.

An interesting observation is that a single GPU with $H^3$ can run a 1 M case, and a 10 M case can be run with two GPUs. This observation shows that long sequences can be processed at extremely low cost, in spite of performance degradation.

*2) Throughput:* Processing larger batch sizes leads to improved throughput. Fig. 6 shows tokens per second (TPS) per request, a metric for throughput mainly determined by the batch size and execution time of the LLM inference system. Throughput of $H^3$ is 1.25x higher in the 1 M case and 6.14x higher in the 10 M case. This means that when applying $H^3$, more tokens can be generated in the same execution time, which directly leads to a reduction in LLM service costs. Furthermore, from this result,

Authorized licensed use limited to: East China Normal University. Downloaded on October 19,2026 at 06:54:01 UTC from IEEE Xplore. Restrictions apply.

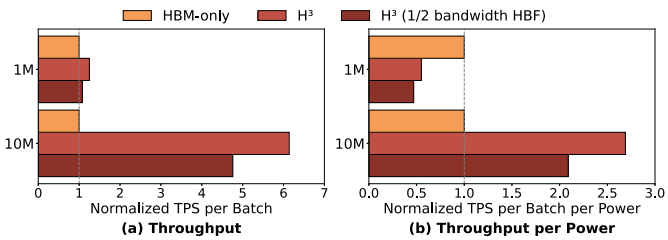



Fig. 6. Comparison of throughput and throughput per power.

it can be predicted that the throughput gap will increase as the sequence length increases.

*3) Throughput Per Power:* To demonstrate that $H^3$ is more cost-effective despite the addition of HBF which consumes more power, we also conducted a throughput per power evaluation. Throughput per power is calculated by dividing the simulated throughput by the total TDP (including GPU TDP, assuming 680 W without HBM or HBF [8]). As shown in Fig. 6(b), despite the higher TDP of the $H^3$-equipped GPU, it achieves up to 2.69x higher throughput per power. These results imply that $H^3$ can be used to cost-effectively handle gigantic read-only LLM inference.

*4) Bandwidth Reduction of HBF:* Although we assume that the HBF has the same bandwidth as the HBM, practical issues during productization may lead to supporting a lower bandwidth. Therefore, we examine the performance impact of halving the HBF bandwidth in the $H^3$. Fig. 6 shows that even with the HBF bandwidth halved, throughput of $H^3$ in the 1 M is still better than that of the HBM-only. Furthermore, in the 10 M case, throughput per power of $H^3$ is still 2.09x higher. For $H^3$, since not all data is stored in HBF, the overall performance is not reduced by the bandwidth reduction rate of HBF.

### C. Silicon Area Overhead of Latency Hiding Buffer

To calculate silicon area overhead, we should first determine the capacity requirement of the LHB. It is calculated by the bandwidth and the access latency of the HBF. To supply data to the GPU with full bandwidth without stopping, double buffering is assumed.

$$Capacity_{LHB} = 2 \times BW_{HBF} \times Latency_{HBF}. \quad (1)$$

In Eq. (1), $Capacity_{LHB}$, $BW_{HBF}$, and $Latency_{HBF}$ denote the size of the LHB, bandwidth per cube of HBF, and access latency of HBF, respectively. Referring to Section IV-A, the $BW_{HBF}$ is 1TB/s. $Latency_{HBF}$ is assumed to be 20 $\mu$s, based on the read access latency of publicly available SLC NAND flash [11]. Therefore, the LHB capacity requirement is 40 MB. Since 20 $\mu$s latency observed in current commodity NAND flash products is expected to improve further when HBF becomes commercially available, the LHB capacity requirement is expected to decrease below 40 MB.

Based on the capacity requirement, the silicon area of the LHB can be calculated. In this paper, we assume the latest 3 nm SRAM, considering the base die process around the time HBF is commercialized in the future. The 3 nm SRAM is known to have a bit density of 0.021 $\mu m^2$ [12]. Therefore, the area of 40 MB SRAM core is 6.72 $mm^2$. Assuming a 20% overhead for the additional circuitry, the total area is estimated to be 8.06 $mm^2$. This is approximately 6.7% of the base die area of 121 $mm^2$, an acceptable value considering the free space of the base die. For reference, 121 $mm^2$ is the base die area of the currently commercialized HBM, and it is expected that the area will increase further in future HBM and custom HBM.

## V. Conclusion and Future Work

This paper proposes $H^3$, a hybrid architecture that effectively integrates HBM and HBF to address the capacity limitations of HBM for long sequence LLM inference. $H^3$ leverages the deterministic nature of LLM workloads to strategically place data: the massive read-only shared KV cache and the model weights are stored in the HBF, while the frequently accessed generated KV caches are stored in the low-latency HBM. To mitigate the inherent high latency of HBF, a latency hiding buffer is introduced. Simulation results demonstrate that the proposed architecture can handle significantly larger batch sizes compared to an HBM-only configuration and achieves up to 2.69x higher throughput per power, proving its potential to realize a cost-effective infrastructure for LLM inference with gigantic read-only data.

$H^3$ is one of several architectures for utilizing HBF. There are various ways to use HBM and HBF together (e.g., hybrid stacking of HBM and HBF), and in certain cases, it may be possible to use HBF alone. Therefore, future work will focus on exploring optimal architectures that can exploit the benefits of HBF and other applications suitable for HBF.

## References

[1] G. Comanici et al., "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," 2025, *arXiv:2507.06261*.
[2] Meta, "Llama 4," 2025. [Online]. Available: https://www.llama.com/docs/model-cards-and-prompt-formats/llama4/
[3] S. Choi, M. Rhee, E. Kim, K. Shin, Y. Joo, and H. Kim, "PNM meets sparse attention: Enabling multi-million tokens inference at scale," *IEEE Comput. Archit. Lett.*, vol. 24, no. 2, pp. 353–356, Jul.-Dec. 2025.
[4] M. Rhee, S. Choi, E. Kim, J. Sim, Y. Joo, and H. Kim, "MoSKA: Mixture of shared KV attention for efficient long-sequence LLM inference," *IEEE Comput. Archit. Lett.*, vol. 24, no. 2, pp. 365–368, Jul.-Dec. 2025.
[5] B. J. Chan, C.-T. Chen, J.-H. Cheng, and H.-H. Huang, "Don't do rag: When cache-augmented generation is all you need for knowledge tasks," in *Proc. ACM Web Conf.*, 2025, pp. 893–897.
[6] A. Dubey et al., "The Llama 3 herd of models," 2024. *arXiv:2407.21783*.
[7] SanDisk, "Memory-Centric AI: Sandisk's high bandwidth flash will redefine AI infrastructure," 2025. [Online]. Available: https://www.sandisk.com/company/newsroom/blogs/2025/memory-centric-AI-sandisks-high-bandwidth-flash-will-redefine-ai-infrastructure
[8] NVIDIA, "Nvidia blackwell architecture technical brief," 2025. [Online]. Available: https://resources.nvidia.com/en-us-blackwell-architecture?ncid=no-ncid,
[9] D. Abts et al., "The GROQ software-defined scale-out tensor streaming multiprocessor: From chips-to-systems architectural overview," in *Proc. IEEE Hot Chips 34 Symp.*2022, pp. 1–69.
[10] R. Patwari, A. Sirasao, and D. Das, "Forecasting LLM inference performance via hardware-agnostic analytical modeling," 2025, *arXiv:2508.00904*.
[11] Q. Wei, Y. Li, Z. Jia, M. Zhao, Z. Shen, and B. Li, "Reinforcement learning-assisted management for convertible SSDS," in *Proc. Des. Automat. Conf.*, 2023, pp. 1–6.
[12] S.-Y. Wu et al., "A 3 nm CMOS finflex platform technology with enhanced power efficiency and performance for mobile SoC and high performance computing applications," in *Proc. Int. Electron Devices Meeting*, 2022, pp. 27.5.1–27.5.4.

Authorized licensed use limited to: East China Normal University. Downloaded on October 19,2026 at 06:54:01 UTC from IEEE Xplore. Restrictions apply.